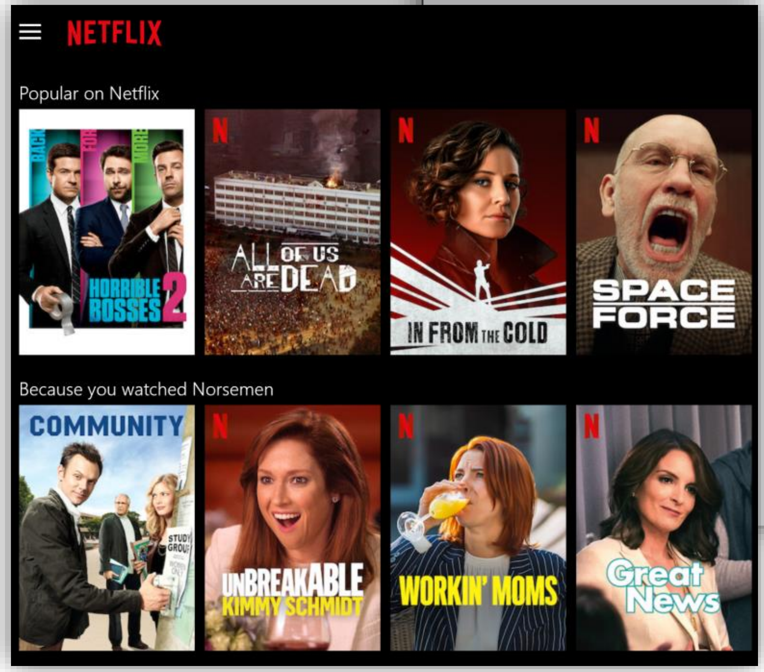
Recommender Systems

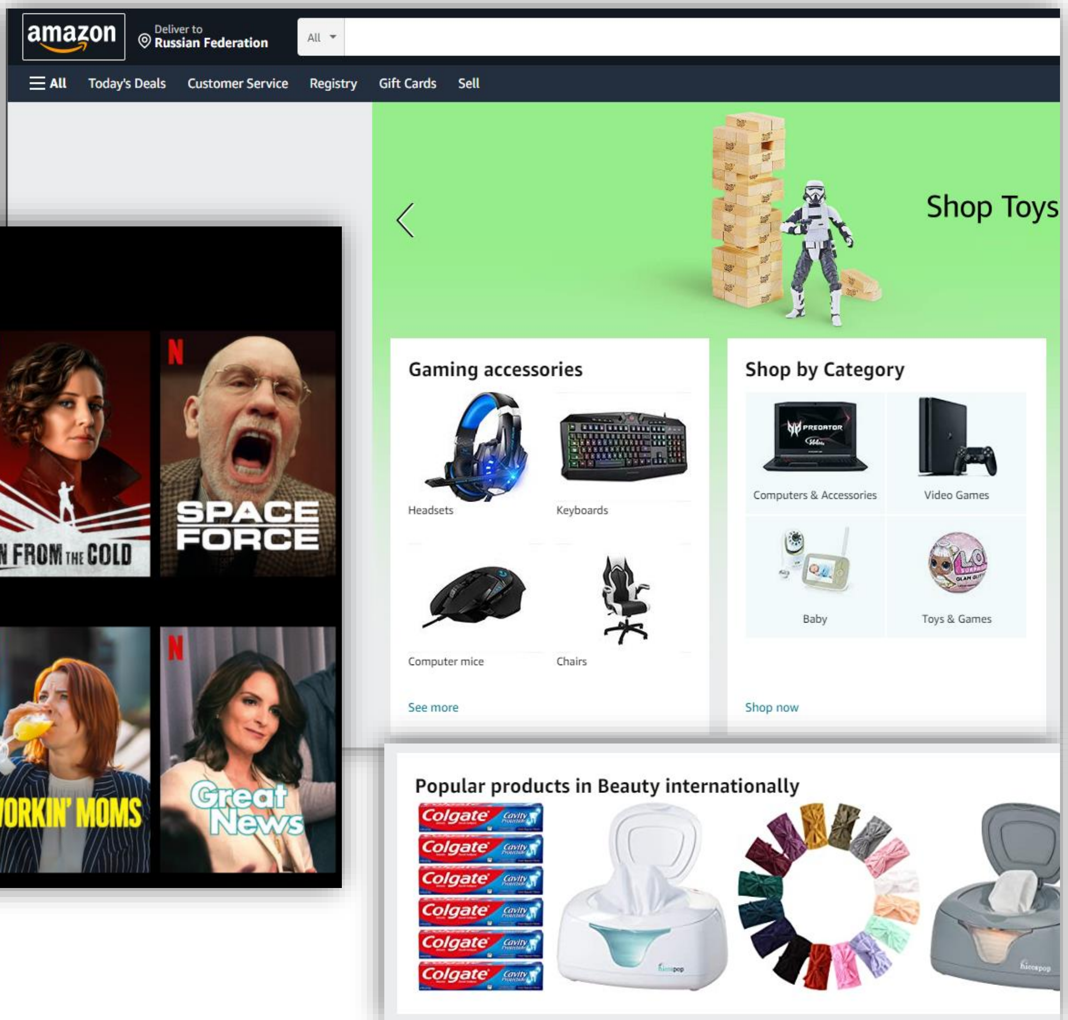
Lecture 2

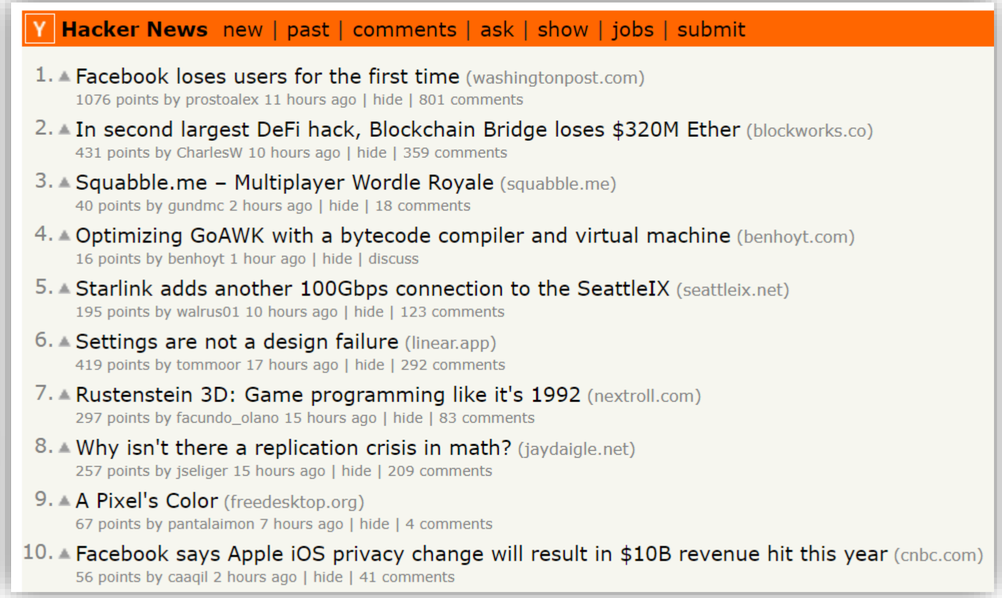
Today's Lecture

- Popularity-based models
- Baseline Predictors
- Practice

Usecases for popularity-based predictions







Popularity-based scoring

Task:

- assign a popularity score based on accumulated feedback from users

- mostly based on heuristics
  - e.g., content "interestingness" ↔ consumption frequency

- typical challenges / issues
  - dynamics and trends
    - e.g., seasonal purchases
  - non-homogeneous distribution
    - e.g., experts vs non-experts
  - attacks and fraud

Examples of user feedback

1. Ratings
2. Upvotes/Downvotes
3. Likes (w/o dislikes)

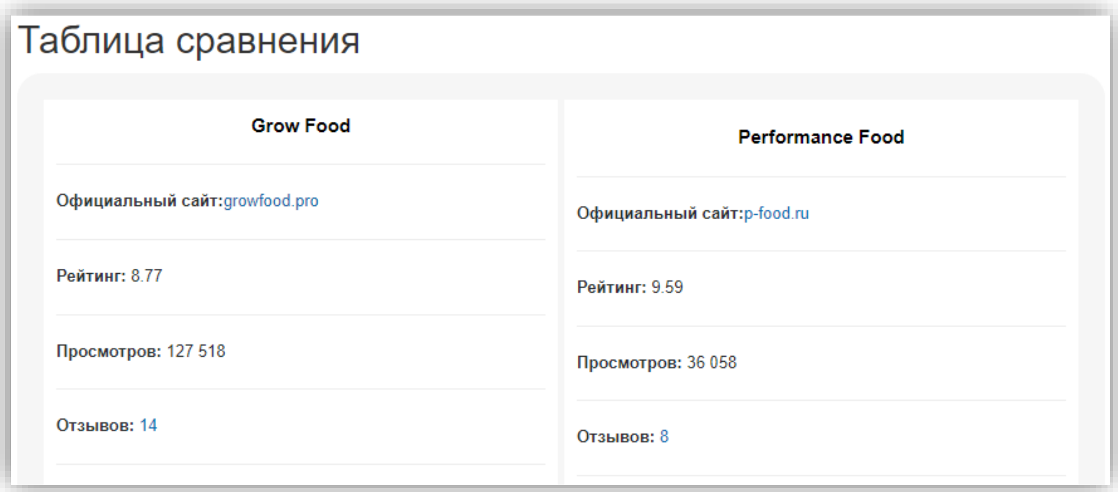
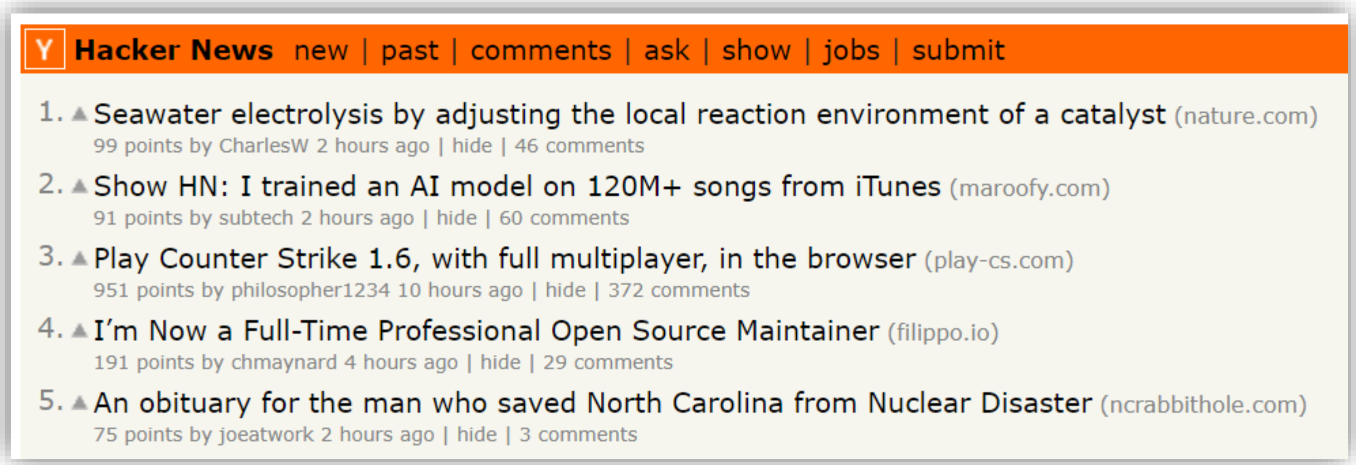


Таблица сравнения

Grow Food	Performance Food
Официальный сайт:growfood.pro	Официальный сайт:p-food.ru
Рейтинг: 8.77	Рейтинг: 9.59
Просмотров: 127 518	Просмотров: 36 058
Отзывов: 14	Отзывов: 8


Y **Hacker News** new | past | comments | ask | show | jobs | submit

1. ▲ Seawater electrolysis by adjusting the local reaction environment of a catalyst (nature.com)
99 points by CharlesW 2 hours ago | hide | 46 comments
2. ▲ Show HN: I trained an AI model on 120M+ songs from iTunes (maroofy.com)
91 points by subtech 2 hours ago | hide | 60 comments
3. ▲ Play Counter Strike 1.6, with full multiplayer, in the browser (play-cs.com)
951 points by philosopher1234 10 hours ago | hide | 372 comments
4. ▲ I'm Now a Full-Time Professional Open Source Maintainer (filippo.io)
191 points by chmaynard 4 hours ago | hide | 29 comments
5. ▲ An obituary for the man who saved North Carolina from Nuclear Disaster (ncrabbithole.com)
75 points by joeatwork 2 hours ago | hide | 3 comments

Rating-based scoring

Popularity-based recommendations

naïve estimation of item j popularity:

$$\mathrm{score}_{\mathrm{POP}}(j) = \frac{1}{|U_j|} \sum_{i \in U_j} r_{ij}$$

- U_j - set of users who rated item j
- r_{ij} - rating of item j provided by user i

Popularity-based recommendations

$$\text{score}_{\text{POP}}(j) = \frac{1}{|U_j|} \sum_{i \in U_j} r_{ij}$$

What are potential flaws?

- insufficient amount of data
  - more data means higher reliability /certainty
  - an item with 100 ratings and average score 4.95 vs item with two 5-star ratings
- different dynamics
  - trends
  - seasonality

In the early days

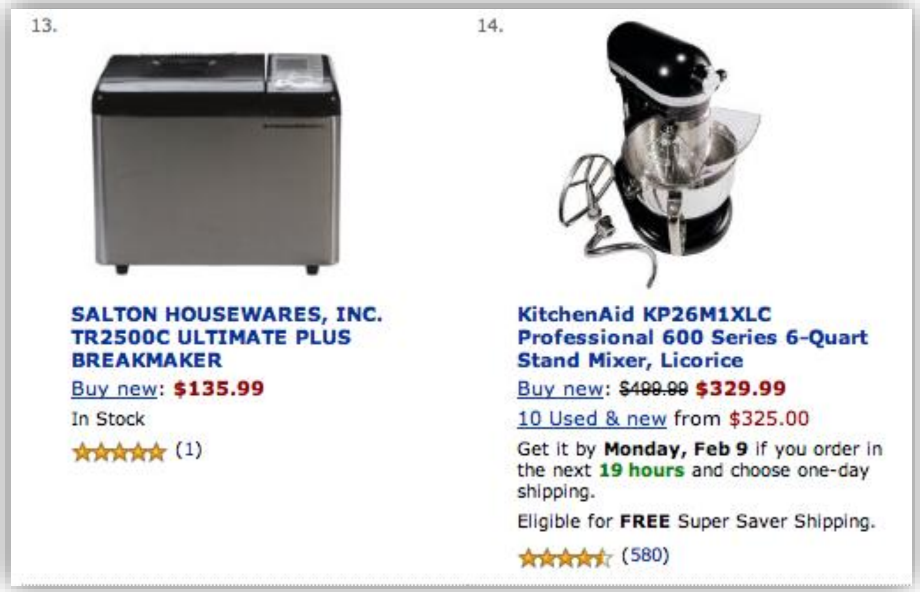
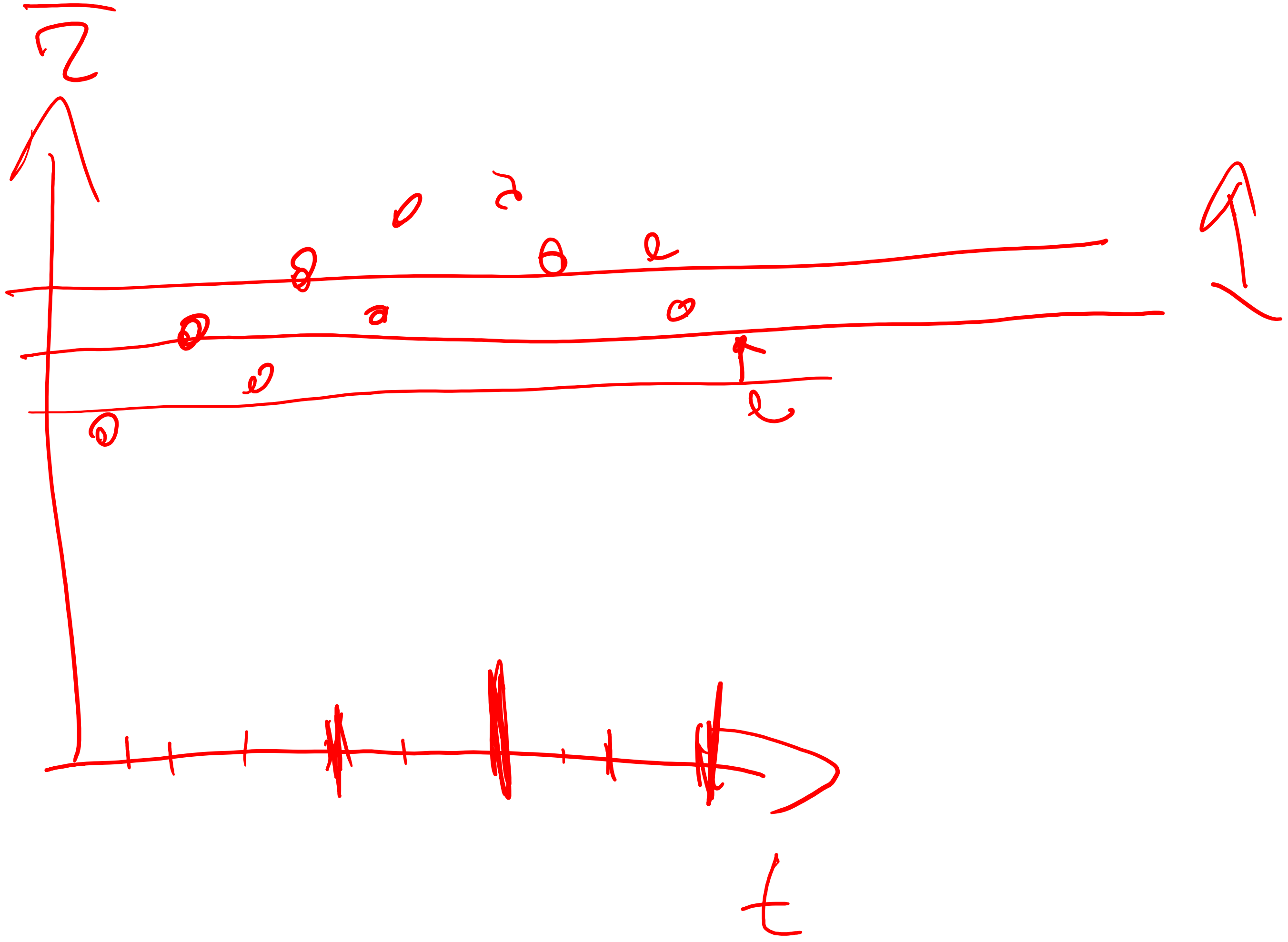
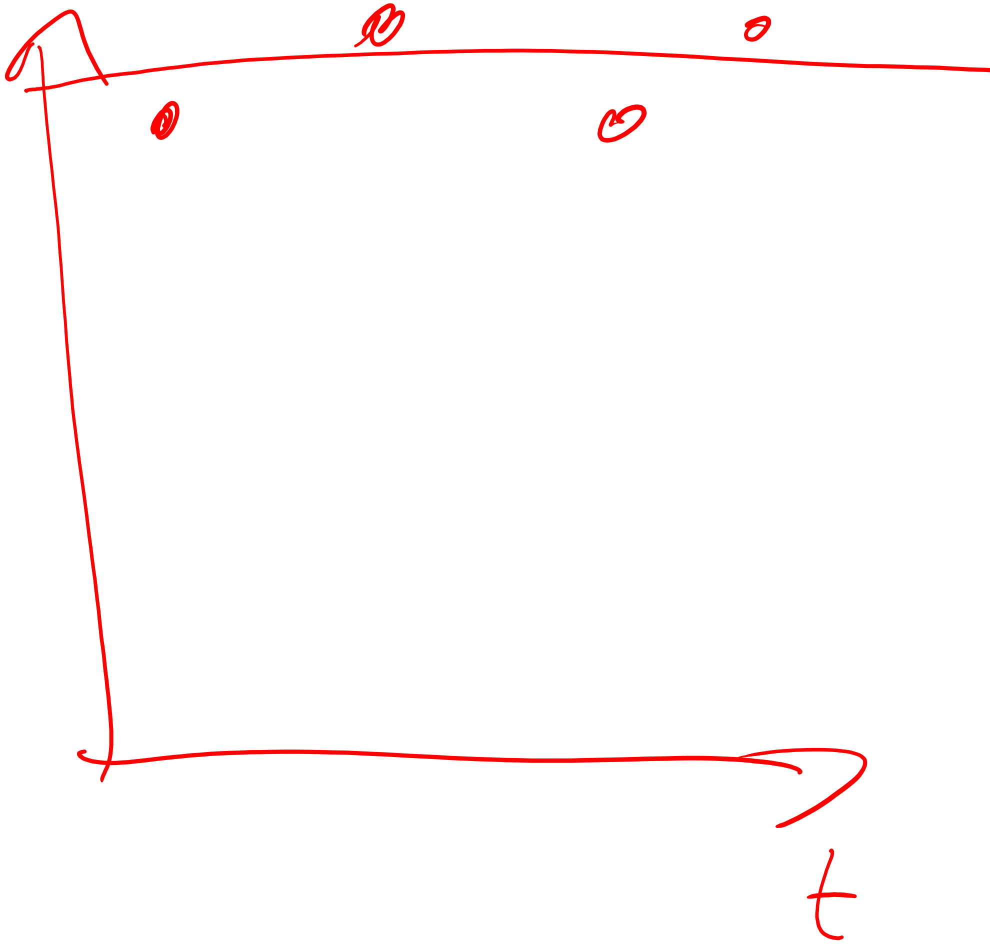


Image from https://www.evanmiller.org/how-not-to-sort-by-average-rating.html

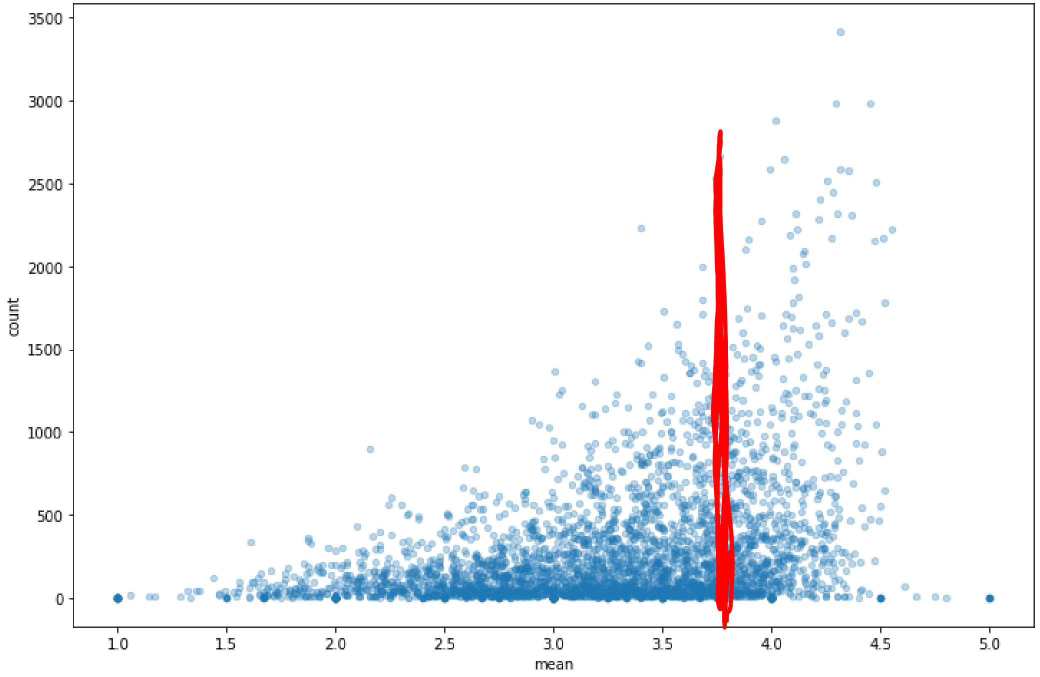
Popular vs niche products





Simple adjustment of popularity

Average rating distribution on Movielens-1M



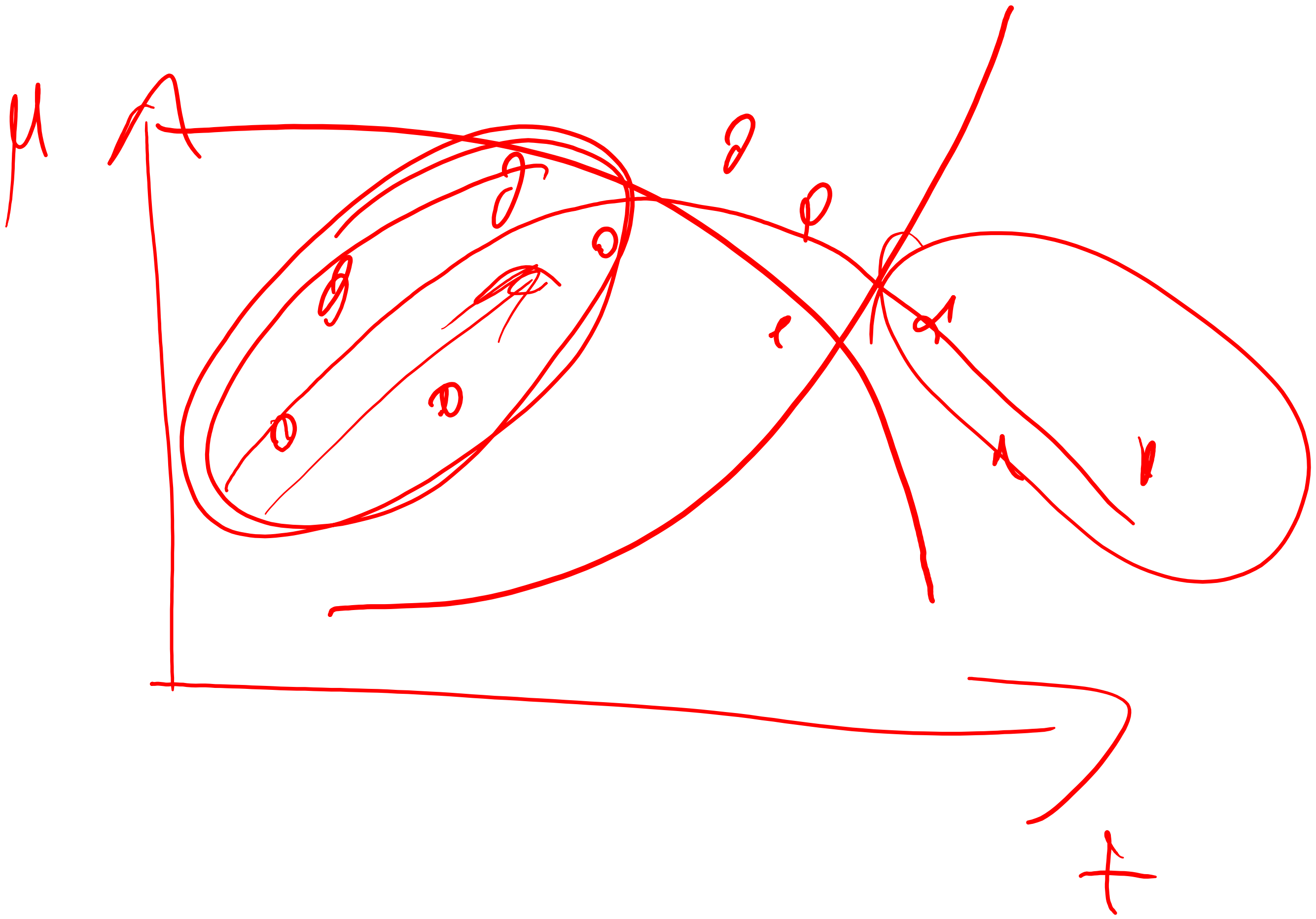
As minimization problem:

$$\sum_{i,j\in O} (z_{ij} - \mu_j)^2 + \lambda \mu_j^2$$

all observed

$$\mu_j = \frac{1}{|U_j| + \lambda} \sum_{i \in U_j} z_{ij}$$

Popularity vs trends in data



Bayesian averaging

combination of prior belief and observed sample average:

$$\mu_{BA} = \alpha \cdot \mu_{PB} + (1 - \alpha) \cdot \mu_{SA}, \qquad 0 \leq \alpha \leq 1$$

assume we're in the future and already have enough data:

$$\mu = \frac{1}{C+n} \sum_{k=1}^{C+n} x_k = \frac{1}{C+n} \sum_{k=1}^{C} x_k + \frac{1}{C+n} \sum_{k=C+1}^{C+n} =$$

$$= \frac{C}{C+n} \underbrace{\boxed{\left(\frac{1}{C} \sum_{k=1}^{C} x_k \right)}}_{PB} + \frac{n}{C+n} \frac{1}{n} \sum_{k'=1}^{n} x_{k'}$$

x_k – sample

C – amount of prior samples

n – amount of newly added samples

Bayesian averaging

$$\mu_{BA} = \frac{C\bar{\mu} + \sum_{k=1}^{n} x_k}{C + n}, \qquad \alpha = \frac{C}{C + n}$$

- For item rating data:

$$\mu_j = \frac{C\overline{r_j} + \sum_{i \in U_j} r_{ij}}{C + |U_j|}$$

U_j - set of users, who rated items j

- too high C → need more observations
- too low C → unreliable estimate

Other possible adjustments

- dampening weights based on recency

$$\sum_j z_{ij} \to \sum_j w_j z_{ij}$$

$$w_j \sim e^{-\alpha t_j} \qquad w_j \sim |t_j - t_0|^{\alpha}$$

- activity of users (bots/trolls or not invested users)

- side information (features, attributes)
  - typically rules-based

Scoring for binary feedback

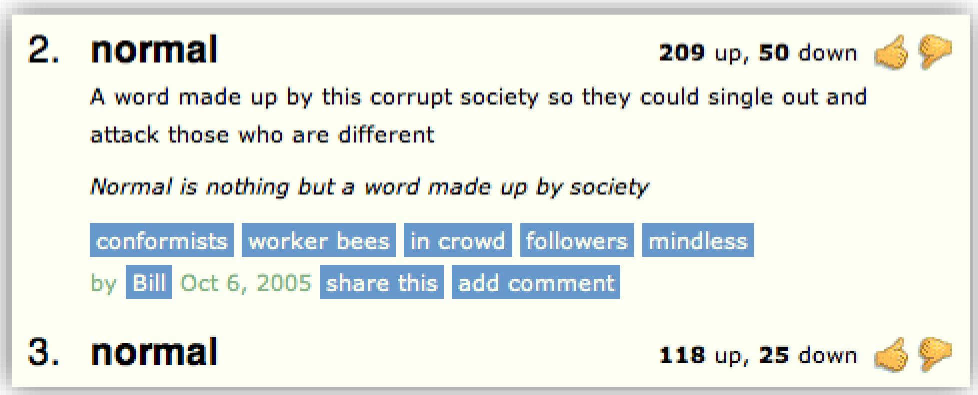
Possible scoring strategies

- Amount of positive feedback

- # positive feedback – # negative feedback

- proportion of positive feedback

In the early days



Urban Dictionary, image credit: https://www.evanmiller.org/how-not-to-sort-by-average-rating.html

Estimating popularity on binary feedback

- lower bound of Wilson score confidence interval*
  - estimate for a Bernoulli parameter
  - estimates the bound with 95% probability given current observations

$$\left(\hat{p} + \frac{z_{\alpha/2}^2}{2n} - z_{\alpha/2}\sqrt{\left[\hat{p}(1-\hat{p}) + z_{\alpha/2}^2/4n\right]/n}\right)/\left(1 + z_{\alpha/2}^2/n\right)$$

  - $\hat{p}$ is the observed fraction of positive ratings
  - $z_{\alpha/2}$ is the $1 - \alpha/2$ quantile of the standard normal distribution
  - n is the total number of ratings

Assumptions: data follows a binomial distribution with

- fixed probability of a success,
- statistically independent trials.

*Example of calculation in python
https://gist.github.com/aditya00kumar/ab81a77258c70f9c3a811f3763a6fb62

Laplace (add-one) smoothing

- $r_{ij} \in \{0, 1\}$ – binary feedback
  $\text{upvotes}_j = \{r_{ij} > 0, i \in U_j\}$

$$\frac{|\text{upvotes}_j| + \alpha}{|U_j| + 2\alpha}$$

$0 \leq \alpha < 1$

- $r_{ij} \in \{-1, 0, 1\}$

- $r_{ij} \in \{1, 2, 3, 4, 5\}$

Some other schemes: Valcarce, D., Parapar, J. and Barreiro, Á., 2016, June. Additive smoothing for relevance-based language modelling of recommender systems. In *Proceedings of the 4th Spanish Conference on Information Retrieval* (pp. 1-8).

Note on popularity / average ratings

- Horror movies ratings are typically lower, even if a user actually likes it.

- "Ghostbusters" Is A Perfect Example Of How Internet Movie Ratings Are Broken.

- IMDb average user rating: 4.1 out of 10, of 12,921 reviewers
- IMDb average user rating among *men*: 3.6 out of 10, of 7,547 reviewers
- IMDb average user rating among *women*: 7.7 out of 10, of 1,564 reviewers

Source: http://fivethirtyeight.com/features/ghostbusters-is-a-perfect-example-of-how-internet-ratings-are-broken/

Baseline predictors

Matrix representation

$R \in \mathbb{R}^{M \times N}$ - rating matrix

r_{ij} - rating of user i for item j

Systematic biases and rating prediction

- Same rating from different users may have very different meanings.
- How can we capture systematic tendencies in rating patterns?

$$b_{ij} = \overline{r_{i:}}$$

$$b_{ij} = \overline{r_{:j}}$$

$$b_{ij} = \left(\overline{r_{i:}} + \overline{r_{:j}}\right) / 2$$

$$b_{ij} = \mu + g_i + f_j$$

Baseline predictors

$$\boxed{r_{ij} \approx b_{ij} = g_i + f_j + \mu}$$

g_i – ***g**enerosity* of user i, i.e. tendency to assign higher or lower rating

f_j – ***f**avoredness of* item j, i.e. how likely it's to be praised or critiqued

μ – global average

tends to capture much of the observed signal

Baseline predictors

$$\operatorname{argmin} \sum_{i,j\in\mathcal{O}} \left(r_{ij} - g_i - f_j - \mu\right)^2 + \lambda\left(\sum_{i\in\mathcal{U}_j} g_i^2 + \sum_{j\in\mathcal{I}_i} f_j^2\right)$$

$\mathcal{O} = \{(i,j)\colon\ r_{ij} \text{ is known}\}$ – set of all known entries (observed data)

$\mathcal{I}_i$ - set of items, rated by user i, $\mathcal{U}_j$- set of users, who rated items j

Iterative updating scheme:

$$\begin{cases} g_i = \dfrac{1}{|\mathcal{I}_i| + \lambda} \displaystyle\sum_{j\in\mathcal{I}_i} (r_{ij} - f_j - \mu) \\ f_j = \dfrac{1}{|\mathcal{U}_j| + \lambda} \displaystyle\sum_{i\in\mathcal{U}_j} (r_{ij} - g_i - \mu) \end{cases}$$

Can be optimized with gradient-based methods as well.

Baseline predictors - decoupled form

$$\begin{cases} g_i = \dfrac{1}{|\mathcal{I}_i| + \lambda_i} \displaystyle\sum_{j \in \mathcal{I}_i} (r_{ij} - \mu) \\[2em] f_j = \dfrac{1}{|\mathcal{U}_j| + \lambda_j} \displaystyle\sum_{i \in \mathcal{U}_j} (r_{ij} - g_i - \mu) \end{cases}$$

λ_i, λ_j - “damping” factors to better capture non-uniformity of data, often set to constant, e.g 25*

* Ekstrand, M.D., Riedl, J.T. and Konstan, J.A., 2011. Collaborative filtering recommender systems. *Foundations and Trends® in Human–Computer Interaction*, *4*(2), pp.81-173.

Baseline estimation – alternative scheme

$$\min_{\boldsymbol{g},\boldsymbol{f}} \underbrace{\sum_{i,j\in\mathcal{O}}}\left(r_{ij} - g_i - f_j\right)^2$$

$|\mathcal{O}| \ll M \cdot N$

$g = \{g_i\}_{i=1}^{M}$ $f = \{f_j\}_{j=1}^{N}$

$$\partial g_{i_0} \Big| -2\sum_{j\in I_i}\left(r_{ij} - g_i - f_j\right) = 0$$

items rated by user i

$R = \begin{cases} r_{ij} & \text{if known} \\ 0 & \text{otherwise}\end{cases}$

matrix of ratings

$\left\{\sum_{j\in I_i} r_{ij}\right\} = Re = R_g$

e - all ones

$S: \; s_{ij} = \begin{cases} 1, & \text{if } r_{ij} \text{ known} \\ 0 & \text{otherwise}\end{cases}$

Sf

$$\underbrace{Re - D_g g - Sf = 0}$$

$D_g = \mathrm{diag}(Se)$

Baseline estimation – alternative scheme

$$\min_{\boldsymbol{g},\boldsymbol{f}} \sum_{i,j\in O} \left(r_{ij} - g_i - f_j\right)^2$$

$$\begin{cases} D_g \boldsymbol{g} + S\boldsymbol{f} = \boldsymbol{b}_g \\ D_f \boldsymbol{f} + S^\top \boldsymbol{g} = \boldsymbol{b}_f \end{cases}$$

S – sparsity pattern of the data, i.e. $s_{ij} = \mathbb{I}\left(r_{ij} \text{ is known}\right)$
$D_g = \operatorname{diag}(S\boldsymbol{e})$ – diagonal matrix of row nnz
$D_f = \operatorname{diag}(S^\top\boldsymbol{e})$ – diagonal matrix of column nnz
$\boldsymbol{b}_g = R\boldsymbol{e}$ – row-aggregated ratings
$\boldsymbol{b}_f = R^\top\boldsymbol{e}$ – columns-aggregated ratings

$$\boxed{\left(D_f - S^\top D_g^{-1} S\right)\boldsymbol{f} = \boldsymbol{b}_f - S^\top D_g^{-1}\boldsymbol{b}_g}$$

Can be effectively solved with e.g. iterative methods (CG, GMRES).

kernel of the system - ?

additional constraint: $\boldsymbol{f} \perp \boldsymbol{e}$
Orthogonality constraint gives natural condition: $\sum_j f_j = 0$

Practice session

Recommended setup:

- dedicated conda-forge env
- VS Code / Cursor (+ python, jupyter extensions)
- numpy, scipy, pandas, matplotlib,
- ipykernel (needed for env management)
- polara

  ```
  pip install --no-cache-dir --upgrade git+https://github.com/evfro/polara.git@develop#egg=polara
  ```
- additionally:
  - numba (for faster execution of python loops if no vectorization available)
  - seaborn
  - lightfm (will need it later, has restrictions on numpy version)
  - scikit-learn